
LIFT: A Counterfactual Framework and Benchmark for Disentangling Genuine Self-Evolution in LLM Agents

Lin Yu*

Tong Han*

Linsheng Zheng*

Abstract

Self-evolving LLM agents continuously accumulate reusable experience *artifacts* — skills, memories, tool libraries, and workflows (SOPs) — produced during interaction and reloaded on later tasks. Yet mainstream evaluation cannot separate *genuine* capability generalization from *pseudo* performance growth introduced by evaluation confounds: training-memory leakage (the agent replaying an earlier task’s output into the next), cross-session context pollution, grader self-preference, and non-reproducible runtimes. As a result, “the agent evolved a pile of artifacts” is routinely mistaken for “the artifacts help downstream.”

We present **LIFT (Loaded Impact on Final Task)**, a counterfactual evaluation framework that makes the *Base* vs. *Loaded* contrast on held-out tasks its single scientific question. LIFT operationalizes this contrast with three mechanisms: (i) **paired A/B runs** of the same held-out task with and without consolidated artifacts; (ii) **strict separation** of warmup (training) tasks from hold-out (unseen) tasks; and (iii) **Docker container snapshots** (docker commit $\rightarrow$ delta image) that solidify the runtime agent state as an artifact-agnostic, cross-machine-reproducible carrier. Together these isolate the confounds above, so that a positive delta on an unseen task is *attributable to the artifacts* rather than to leakage, context pollution, or environment drift. A **three-layer decoupling** (pipeline / runtime adapter / evaluation kernel) hosts a range of mainstream self-evolving agent runtimes behind four hooks; a **work-agent + judge-agent review loop** scores each task; and multi-level concurrency plus Langfuse trace backfill make every run auditable in both *conclusion* and *process*. We release a self-contained **benchmark of 14 scenes (84 tasks)** built on a two-phase schema with a deliberate $\sim 75\%/25\%$ train-test requirement overlap designed to expose over-fitting to literal training requirements. LIFT provides a standardized, attributable pipeline for empirical research on self-evolving agents and makes precise the methodological gaps in existing agent benchmarks. Code is released at <https://github.com/FeiZhuNiU-INFJA/LIFT> and the benchmark at <https://huggingface.co/datasets/FeiZhuNiU-INFJA/EALE>.

Preprint status. This is a v1 preprint. §6 reports population-ratio Δ from a partial cross-runtime sweep (11 runtimes across RQ1–RQ5); remaining runtimes and full-suite repeats will be released in v2.

*Equal contribution; author order is arbitrary and does not reflect seniority or contribution weight. All authors are independent researchers.

1 Introduction

1.1 From model evaluation to agent evaluation

LLM evaluation began as “feed a prompt, check the output.” Agents shattered that: a single task becomes a long chain of *planning* → *tool calls* → *file-system operations* → *multi-turn feedback* → *final deliverable*, and the model is only one node. An agent that appears to “answer correctly” may do so because (i) the task itself is flawed (the ABC Checklist [Zhu et al., 2025] reports overestimation of up to 100% on ill-posed items), (ii) it *replays* the previous task’s output into the next task (pseudo-evolution), (iii) the grader is the same model family (self-preference bias), or (iv) environment fidelity is too low (a safety-benchmark taxonomy [Fu et al., 2025] reports Kendall’s $W \approx 0.10$ — near-random cross-benchmark safety rankings).

1.2 Self-evolving agents magnify the problem

Over the past year, self-evolving agents accumulate reloadable artifacts — skills, memories, tool libraries, workflows (SOPs) — during interaction. Commercially, “an agent that keeps learning” is the headline; academically, the four-component loop (System Inputs → Agent System → Environment → Optimisers) has crystallized. But a blunt empirical fact stands out: on SkillsBench’s 86 tasks [Li et al., 2026], *curated* skills yield +16.2pp on average, while *self-generated* skills are on average useless or harmful. Hence:

“The agent evolved a pile of artifacts” $\neq$ “the artifacts help downstream.”

These must be evaluated separately, and the method must be a **causal contrast** — not how high the evolved agent scores in absolute terms, but the score difference on *the same task, same agent, same tool set, artifacts loaded vs. not*.

1.3 Three fault lines in current evaluation

Surveying 32 open-source works (surveys / benchmarks / SDKs) along the axis “can it answer *do the artifacts help?*”, we find three shared fault lines:

Table 1: Three shared fault lines in current agent evaluation.

Fault line	Symptom	Representative work
No loaded/unloaded contrast	Benchmarks measure a bare agent’s pass rate; no artifact-dimension A/B	AgentBench, GAIA, SWE-bench, REAL, τ -bench
Train and test tasks entangled	Agent learns on Q1..Qn then is tested on the same Q1..Qn — cannot separate “learned” from “memorized”	most episodic self-evolution setups
Runtime not reproducible / not isolated	Multi-step FS ops, concurrency, missing trace correlation → unstable deltas, huge variance across repeats	most host-machine eval scripts

The third is the most underrated. Once evaluation must be *run repeatedly* — for commercial artifact certification, for iterating an evolution mechanism, for comparing runtimes — reproducibility flips from nice-to-have to hard requirement. REAL [Garg et al., 2025] centered exactly on making deterministic simulation a first-class citizen of web-agent evaluation, evidencing the community’s weight on this.

1.4 Contributions

We propose **LIFT**, whose thesis is one sentence:

To evaluate a self-evolving agent, ask “does it do better on *unseen* tasks after loading its artifacts?” — not “how high does it score after running through all tasks?”

Concretely we contribute:

1. **A scientific protocol** — train/test separation (`warmup_tasks` / `holdout_tasks`) + same-task paired runs (baseline / evolved) + a work-judge review loop (§3).
2. **An engineering carrier, released** — container snapshots (`docker commit` → delta image) as the artifact-agnostic form of evolved state; each holdout task starts an isolated container; runtime / pipeline / evaluation are decoupled into three layers (§3.3, §3.5). We release the framework code, adapters for mainstream self-evolving agent runtimes, container images, and datasets, so the protocol is reusable, criticizable, and comparable.
3. **Repeatability & observability** — multi-level concurrency and pre-chat + Langfuse trace backfill, so both conclusion and process are re-queryable (§3.6–3.7).
4. **A self-contained benchmark suite** — 14 scenes / 84 tasks on a two-phase schema with a deliberate 75%/25% train-test requirement overlap (§4).

2 Related Work

General agent benchmarks. AgentBench [Liu et al., 2023], GAIA [Mialon et al., 2023], REAL [Garg et al., 2025], τ -bench / τ^2 -bench [Yao et al., 2024, Barres et al., 2025], SWE-bench [Jimenez et al., 2023], and harness-comparison benchmarks answer “can the agent do the job on a static task set.” They are LIFT’s *task source*, not its contrast object. Some already do cross-harness comparison and tiered evaluation (script verification / point-wise verification / pair-wise comparison), a design we reuse — but none reach the artifact-dimension causal contrast.

Artifact benchmarks. SkillsBench [Li et al., 2026], EvolveTool-Bench [Kaliyev and Maryanskyy, 2026], and memory benchmarks (EvoMemBench [Wang et al., 2026], Evo-Memory [Wei et al., 2025]) begin to probe the artifact dimension but each binds to one artifact form (skill / tool / memory). LIFT’s **artifact-source-agnostic** stance abstracts over them: whether the artifact is a skill, a memory, a tool library, or a workflow, the verification logic is *Base vs. Loaded*.

Lifelong / sequential learning. LifelongAgentBench [Zheng et al., 2025] and SEA-Eval [Jiang et al., 2026] provide learning-curve metrics (FWT / BWT / RecoveryRate). LIFT positions these as **evolution-specific diagnostics**: if the Base-vs-Loaded test is conclusive, no curve is needed; when it is not, learning curves help diagnose forgetting vs. non-transfer. This differs from prior work that treats them as primary metrics, and it avoids coupling evolution-mechanism complexity into the main evaluation path.

Evaluation methodology. The ABC Checklist [Zhu et al., 2025], Agent-as-a-Judge [Zhuge et al., 2024], and MAJ-Eval [Chen et al., 2026] answer “how do we grade reliably.” LIFT’s review loop borrows Agent-as-a-Judge’s *tool-augmented verification*: the judge calls real tools to verify outputs rather than grading on surface text (LIFT sandboxes it in a sibling container; §2.1).

Evaluation SDKs. DeepEval [Confident AI, 2025], Opik [Comet ML, 2025], and Promptfoo [Promptfoo, 2025] target LLM-API-layer developer evaluation at the granularity of `prompt` × `model`. LIFT’s granularity is *in-container agent* × *holdout task* × *baseline/evolved* — a different layer. Langfuse [Langfuse, 2025] plays Opik’s *trace* role in LIFT — we use only its trace capability, not its evaluator, because the evaluator role is filled by the work-judge loop.

Safety evaluation. OS-Harm [Kuntz et al., 2025] and RAS-Eval [Fu et al., 2025] flag that loading artifacts introduces new safety risk. LIFT keeps SafetyRegression as an optional cross-cutting dimension (off by default, to avoid entangling it with functional artifact evaluation); a full safety protocol is orthogonal future work.

2.1 LIFT vs. existing work: where LIFT fills the gaps

The five paragraphs above sketch *who does what*. This subsection zooms into the five works closest to LIFT along the axis “can this answer *do the artifacts help?*”. Each contrast is expressed as: *what the work does well, what design LIFT reuses, and what LIFT adds that this work does not*. A wider nine-dimension matrix appears in Appendix B.

SkillsBench [Li et al., 2026] — motivating evidence, not a substitute. SkillsBench’s three-condition design (*no skill / curated skill / self-generated skill*) yields the fact that motivates LIFT (§1.2): generation quality is not implied by generation activity, so evaluation must contrast *loaded* vs. *not*. LIFT reuses this ternary logic but generalises the artifact form: SkillsBench binds to skill packs while LIFT is **artifact-source-agnostic** — whatever the runtime writes to disk during warmup is committed into a delta image and reloaded, so skill, memory, tool library, and workflow all pass through the same protocol. SkillsBench cannot answer “does memory help?” because memory is not in its scope; LIFT can.

LifelongAgentBench [Zheng et al., 2025] — learning curves are diagnostic, not primary. LifelongAgentBench is the closest lifelong-learning benchmark and centres FWT / BWT / RecoveryRate as its primary metrics — capturing forward transfer, catastrophic forgetting, and recovery. LIFT considers these *necessary diagnostics but insufficient primary metrics*: a positive FWT on a curated task ordering does not answer whether the artifact helps on *arbitrary* unseen tasks, and BWT is defined only along the ordering used at training. LIFT instead makes **Base vs. Loaded on held-out tasks** the primary contrast (P1–P2 in §3.1) and demotes FWT / BWT to optional diagnostics reserved for cases where P1 is inconclusive. This inversion is intentional: it lets a benchmark author decide train / test separation once (in the suite JSON) rather than every time a curriculum is redesigned.

Agent-as-a-Judge [Zhuge et al., 2024] — LIFT inherits tool-augmented verification, but sand-boxes it. Agent-as-a-Judge shows that a judge with tool access verifies multi-step agent outputs far more reliably than surface-text LLM-as-a-Judge. LIFT’s work $\leftrightarrow$ judge review loop (§3.4) directly adopts this stance: the judge is a real agent in the *same* runtime, capable of executing tools to verify (e.g., open the artifact file, run the reproduction script). Where LIFT adds design: (i) the judge runs in a *sibling container* of the work agent, not inside the same session — so file-system, memory, and process state cannot leak from work to judge, closing a self-preference pathway that shared-session LLM-as-a-Judge suffers from; and (ii) both work and judge share the same Langfuse `session_id`, giving one navigable trace tree per task while keeping state strictly isolated.

EvolveTool-Bench [Kaliyev and Maryanskyy, 2026] — static tool quality is orthogonal to loaded impact. EvolveTool-Bench evaluates *tool libraries* on four dimensions (correctness, reusability, composability, regression safety), showing that tools which work on the task that produced them frequently fail to generalise. This is a valuable *static* check on a tool artifact — but static quality does not imply downstream lift. LIFT is deliberately positioned one layer above: it asks whether the runtime that produced the tool (plus everything else in the delta image) benefits a held-out task. The two are complementary: EvolveTool-Bench diagnoses *which tool is broken*, LIFT diagnoses *whether loading any of the tools together helps*. A tool that passes EvolveTool-Bench can still produce a null delta on LIFT; a null delta on LIFT is a signal that the static check missed the composition-level failure that only shows up under real load.

REAL [Garg et al., 2025] — reproducibility as a first-class citizen. REAL (NeurIPS 2025 Oral) centres on making deterministic simulation of web tasks a first-class evaluation citizen: 11 real websites replicated as fixed sandboxes, 112 tasks with programmatic verifiers, so that a repeat run gives you a comparable score even six months later. LIFT applies the same principle in a different scope: not task-level determinism (LLM sampling variance remains) but **artifact-level reproducibility** — the exact delta image reloaded on any machine reproduces the same evaluated agent state, and Langfuse trace backfill re-hydrates the process record. Where REAL’s contribution is “the environment does not drift,” LIFT’s contribution is “the artifact does not drift.” We view REAL as the closest methodological ancestor for the reproducibility principle P4, though the two answer different scientific questions.

3 The LIFT Framework

LIFT = Loaded Impact on Final Task. The scientific question: **after loading artifacts, is there positive impact on the final (holdout) task?**

3.1 Design principles

We treat the survey conclusions as hard constraints that all engineering must obey:

Table 2: Design principles P1–P5 that all LIFT engineering must obey.

Principle	Meaning
P1 Single causal contrast	The only variable under test is “loaded vs. not”; everything else held constant
P2 Train/test separation	The tasks used to evolve (warmup) and the tasks used to test (holdout) must not overlap
P3 Artifact-source agnostic	Whether the artifact is a skill / memory / tool library / workflow, the evaluation logic is identical
P4 Reproducible runtime	The same run_id re-run on any machine should produce a consistent structure (including traces)
P5 Optional evolution diagnostics	Learning curves (FWT / BWT) are used only when P1 is inconclusive

3.2 Protocol layer: what a “LIFT run” is

A **Suite** consists of two task groups — this is P2 enforced at the data-structure level:

```
SuiteSpec
|-- warmup_tasks[] <- trigger artifact production (learn / evolve)
'-- holdout_tasks[] <- final tasks, the tasks under test
```

warmup and holdout are **explicit** separate fields in the suite JSON; the runtime no longer slices by question index.

Control groups. Each holdout task is run twice, forming one TaskRun:

```
TaskRun
|-- baseline PhaseRun <- clean container from base image, no artifacts
'-- evolved PhaseRun <- clean container from delta image, with artifacts
```

Key engineering trade-offs.

- **Warmup container orchestration is configurable.** In a single-agent evolution scenario, all warmup tasks share one container by default (`parallel_single`; continuous file-system state is a natural requirement of the evolution plugin). The framework also supports `parallel_multi` for scenarios where artifacts should live outside the container (e.g., an external memory service): each task gets its own container, artifacts land in the external service, and `docker commit` degenerates to a no-op. Both forms are transparent to the upper pipeline.
- **Holdout starts a new container per task.** Unlike warmup, this is a protocol-level hard constraint — baseline must be an *uncontaminated* environment, and per-task workspaces must be isolated. Hence `HoldoutContainerPolicy` (`src/lift/policies/container.py`) offers only `SERIAL_MULTI` / `PARALLEL_MULTI` “multi-container” forms, with no single-container option.
- **All holdout tasks share one delta.** The artifact is a suite-level constant and must not vary across holdout tasks.

Report hierarchy.

```
EvalReport
'-- runs[] <- --repeat iterations
    '-- suites[] <- --suite multiple suites
        '-- tasks[]
            |-- baseline: PhaseRun
            '-- evolved: PhaseRun
```

3.3 Abstraction layer: runtime / pipeline / eval decoupling

LIFT’s code follows three abstraction layers, each with a single concern:

```

CLI / Pipeline <- slice tasks, loop, concurrency, write report (doesn't know what
  OpenClaw is)
  |
  v
AgentRuntimeAdapter <- container, artifact materialization, how to chat (doesn't
  know what holdout / repeat is)
  |
  v
lift/eval (work + judge) <- per-task review loop (doesn't know what Docker is)

```

The direct payoff: **integrating a new runtime is near-zero-cost**. Registered runtimes include:

Table 3: Registered runtimes and their evolve behavior after warmup.

runtime	image	evolve behavior
openclaw	base image, no evolution plugin	no-op after warmup; only docker commit
openclaw_with_openspace	with-openspace image (OpenSpace MCP skill hub)	reuses base warmup / commit flow; MCP-side skill hub is exercised during warmup
openclaw_with_agentmemory	with-agentmemory image	container-local agentmemory server on :3111 (offline embedding); bridge network; base warmup / commit flow captures /root/.agentmemory
genericagent / genericagent_active_evolve	lift-genericagent:latest	baseline file I/O; active variant does reflection chat
hermes	lift-hermes:latest	Hermes review flow writes /opt/hermes-state, then commit
hermes_with_openspace	lift-hermes-with-openspace:latest	Hermes review flow + OpenSpace MCP registered in config.yaml; commit
hermes_with_agentmemory	lift-hermes-with-agentmemory:latest	Hermes review flow + agentmemory provider (container-local :3111, bridge network); commit
openhuman	lift-openhuman:latest	Rust JSON-RPC runtime; long-term memory / wiki paths enter the delta
openhuman_with_agentmemory	lift-openhuman-with-agentmemory:latest	config.toml [memory] backend=agentmemory routes to container-local :3111; bridge network; commit captures /root/.agentmemory
evoscientist / evoscientist_active_evolve	lift-evoscientist:latest	baseline captures natural state change; active variant triggers EvoMemory AutoSkills, then commit

MCP note. OpenSpace variants are only added to MCP-capable runtimes (OpenClaw, Hermes). GenericAgent (fixed atomic tools) and OpenHuman (no mcp_servers field) are not MCP clients, so they do not receive an OpenSpace variant.

A new runtime implements four hooks — resolve_docker_image / start_container / worker_judger_factory / evolve_after_warmup — on the container base class ContainerAgentRuntimeAdapter (src/lift/adapters/container/adapter.py), which reuses the entire docker commit / holdout-orchestration logic. This directly realizes P3: different runtimes place artifacts in the image / group memory / file system, all transparent to the pipeline.

Counter-example (design origin). An early host-machine implementation bound OpenClaw directly into the eval script. When we tried to integrate a second runtime (Hermes), the whole pipeline had to be rewritten — the direct motivation for the current three-layer abstraction.

3.4 Per-task kernel: the work + judge review loop

The single task is LIFT’s minimal scoring unit. We use an *execute* → *review* → *feedback* → *retry* loop.

Why a loop, and why this shape. Real-user interaction is neither one-shot nor rubric-driven. A user rarely writes their full acceptance criteria up front; they issue a query, look at the deliverable, and only then surface an unmet requirement — *one at a time*, in natural language, in the tone of “oh, and I also wanted X.” An agent operating under this loop must not only produce a plausible first draft, but also *interpret and act on partial, unordered feedback* — a strictly harder skill than answering a fully-specified prompt. Any evaluation that collapses this to a single “answer → score” step loses the most discriminative signal about artifact usefulness: whether the evolved agent needs *fewer* clarification rounds than the base. LIFT therefore mirrors the interaction rather than the grading rubric:

- **Query only, checklist hidden.** The agent receives what a user would actually type; the ground-truth `content_reqs` list never enters the agent’s context (§4.2).
- **Judge as the user, not a rubric adjudicator.** The judge role-plays the person who wrote the query, compares the deliverable against the hidden checklist, and returns a natural-language reason that surfaces *the top unmet requirement(s)* only — not the full diff, reconstructing the “user suddenly remembers one more thing” dynamic.
- **Feedback becomes the next prompt.** That reason is fed back verbatim as `current_prompt`, so the follow-up turn is indistinguishable in shape from a first turn — another human-style utterance, no metadata leak, no structured tool-call trace injected on the user’s behalf.
- **Bounded by a shared `max_turns`.** The loop terminates when the judge marks success or the budget is exhausted; baseline and evolved share the same budget (rationale in decision 3 below).

The code block below encodes exactly this loop:

```
run_task:
  while turn < max_conversation_turns:
    work_result = work_agent.chat(current_prompt)
    judge_result = judge_agent.chat(...) -> JSON {success, reason, score}
    if judge_result.success: return True
    current_prompt = judge_result.reason # disclose the top unmet requirement,
    human-style
  return False
```

Design decisions:

1. **The judge runs in a sibling container of the same runtime** — started from the same image and workspace as the work container, but sharing neither its file system nor its process space. This simulates a real user reviewing, introduces no cross-model bias, and lets the judge call real tools to verify outputs (Agent-as-a-Judge’s *executable verification*). The physical split also prevents the work agent from reading the judge’s tool state, and — combined with §3.5 — keeps any reviewing artifact out of the delta.
2. **First-round and final pass rates are reported separately** — `FirstRoundPassRate` reflects “artifacts let the agent get it right in one shot”; `FinalPassRate` reflects “artifacts + feedback loop.” Good artifacts show up mainly in the former.
3. **baseline and evolved must share `max_turns`** — otherwise evolved wins by getting two more feedback rounds, not by artifact merit.
4. **Tokens include the judge’s consumption** — if Loaded retries less, the judge’s token savings count toward the artifact’s contribution.

Scoring. The judge returns a single scalar score = (satisfied requirements)/(total requirements) (score = 1 on success), together with `success` and the natural-language `reason`. There is no rule/rubric weighted blend — the score is one judge call. Content requirements and trajectory requirements are scored on two separate lines (§4.2).

3.5 Artifact materialization: why `docker commit`

The carrier form of the artifact is an engineering choice that *looks incidental but determines reproducibility*. We considered three options:

Table 4: Three candidate artifact carriers and why LIFT chooses container snapshots.

Option	Pro	Fatal con
Host-machine toggle load	simple	state depends on host FS; concurrency cross-contaminates; not replayable across machines
Structured export (YAML/JSON artifact pack)	readable, versionable	hard to cover FS-level evolution (OpenClaw’s entire <code>~/ .openclaw/ tree</code>)
Container snapshot (<code>docker commit</code>) ✓	full FS capture; naturally cross-machine replayable; naturally concurrency-isolated	delta images have size; need cleanup

LIFT chooses `docker commit`. A full run’s timeline:

```
Pipeline -> Adapter: warmup tasks (all warmup_tasks[])
Adapter -> Warmup container: start one container, run tasks continuously (default parallel_single)
Warmup container: evolve_after_warmup (in-container learn review)
Warmup container: docker commit -> delta image
Adapter: delete warmup container

for each holdout task:
  Pipeline -> Adapter: baseline -> new container from BASE image -> run + score
  Pipeline -> Adapter: evolved -> new container from DELTA image -> run + score

Pipeline: write report.json; on suite end, docker rmi the delta
```

Implicit constraints:

- **Delta is a suite-level temporary artifact** — `docker rmi`’d when the suite finishes, never polluting the local image list.
- **Holdout workspace must be explicitly seeded** — to avoid a first-run agent asking for a name/emoji and confusing scoring.
- **`docker commit` operates on the work container only** — the judge sibling (§3.4) is torn down without commit, so no reviewing artifact can leak into evolved; tool counts, tokens, and `evolve_after_warmup` hooks are likewise scoped to the work side.
- **External-artifact runtimes** use the same interface `DeltaRef(image_tag=base_image, owned=False)` (`owned=False` skips `docker rmi`), unifying “artifacts live externally” under one abstraction.

3.6 Concurrency & isolation: making “run it repeatedly” cheap

As evaluation goes from “produce one report” → “iterate continuously” → “commercial artifact certification,” the cost of repeated runs becomes increasingly sensitive. LIFT adds concurrency + isolation at three levels:

Each level of concurrency rests on container isolation: every holdout task gets its own container, workspace subdirectory, and ephemeral port; cleanup is centrally tracked by a `SuiteRunResources` registry. Failure isolation prevents cascades; first-pass failed cells are retried once globally, phases/-tasks retry once each, and the chat layer retries provider errors $5\times$ / judge-JSON parse $8\times$.

Table 5: Concurrency and isolation levels with their flags and defaults.

Level	Flag	Default
Between matrix cells (repeat $\times$ suite Cartesian product)	<code>-max-parallel-suites</code>	3
Between tasks (within a phase)	<code>-max-concurrent-tasks</code>	no cap
Between phases (within a task)	<code>-holdout-phase-policy</code>	parallel
Warmup container policy	<code>-warmup-container-policy</code>	parallel_single
Holdout container policy	<code>-holdout-container-policy</code>	parallel_multi

3.7 Observability: an execution-time + post-processing dual chain

LIFT writes its report in two passes:

Table 6: The dual-chain report writing: execution-time and post-process passes.

Phase	Content	File
Execution	conclusion: success, score, session, workspace paths	<code>report.json</code>
Post-process	Langfuse trace backfill, comparison CSV, trajectory judging, HTML report	<code>*_backfilled.json</code> , <code>*_comparison_metrics.csv</code>

Why two passes: at execution time, traces are still in the Langfuse worker queue — synchronously waiting would slow the main path; and post-processing can be re-run independently (`-evaluate-only`) to diagnose or add new metrics. Execution-time `emit_pre_chat_state` stamps `session_id` and phase metadata before each chat; post-processing `stitch_phase_langfuse_traces` merges the agent’s `*_agent` trace with the plugin trace by `session_id` + time window into `PhaseRun.langfuse`. This contract guarantees **the report can open the trace** to show what actually happened on that task.

3.8 Metric system

- **Required:** $\text{DownstreamPassDelta} = \text{PassRate}(\text{Loaded}) - \text{PassRate}(\text{Base})$, $\text{FirstRoundPassRate}$, FinalPassRate , AvgAttempts , $\text{Outcome}_i = \text{score}$ (the judge’s single completion ratio = satisfied/total, 1 on success), TotalTokens (incl. judge), TotalLatency .
- **Efficiency (primary for evolution):** $\text{impr_metric} = \text{evolved_metric} / \text{baseline_metric} \times 100\%$ over {attempts, tool calls, tokens} — an evolution that reaches the user’s requirements in fewer interactions on similar tasks is beneficial; the greater the reduction, the better the mechanism.
- **Sequential:** Pass@k , RecoveryRate .
- **Evolution-specific (optional):** attribution triplet *No-Evo / Only-Products / Evo-On*; curve diagnostics FWT / BWT.
- **Cross-cutting:** static $\text{DistillateConflictRate} / \text{SafetyConcernCount}$, dynamic SafetyRegression , cost $\text{ColdStartTTV} / \text{EvolutionROI}$.

LIFT itself only emits a **structured execution record (Pydantic)**; which metrics are *required* is decided by the benchmark — hence the benchmark is described as a distinct component, orthogonal to the protocol.

3.9 Contrast with replay mode

An early replay mode (all-suite baseline $\rightarrow$ one evolve $\rightarrow$ all-suite evolved) is not the main path; it fits only as an ablation:

LIFT picks extrapolation as the **main** protocol precisely to exclude the second fault line of §1.3.

Table 7: LIFT’s holdout-extrapolation protocol versus the diagnostic replay mode.

	LIFT (this paper)	replay
Focus	loaded contrast on holdout final	run all tasks once before and once after evolution
Artifact stage	warmup $\rightarrow$ Δ image, then test	evolve once after all-suite baseline
Risk	none	Q5 baseline may be polluted by Q1 . . Q4 state; Q5 evolved may directly reuse Q1 . . Q4 outputs
Scientific question	do artifacts help on <i>unseen</i> tasks (extrapolation)	per-task evolution magnitude (interpolation , diagnostic)

4 The LIFT Benchmark Suite

This section is self-contained: the benchmark design (schema, task-authoring principles, rubric design, the 14 scenes) is described here rather than deferred to a companion paper.

4.1 Three-layer structure

The dataset is organized in three layers — **total dataset** $\rightarrow$ **scene datasets** $\rightarrow$ **train/test tasks** — stored human-readably as folders + task markdowns and compiled by `python-msrc.cli.preprocess` into machine-readable suite JSON (Suite in `src/models.py`):

```
<scene>/
|-- train/ # warmup tasks (artifact evolution)
| |-- q1_<shortname>/ {q1_<shortname>.md, q1_materials/}
| '-- ... # 4 tasks
|-- test/ # final / holdout tasks (LIFT paired contrast)
| |-- q5_<shortname>/ ...
| '-- q6_<shortname>/ # 2 tasks
'-- skills/ # optional, scene-level skill
```

The compiled Suite explicitly separates `warmup_tasks []` ($\leftarrow$ train/) from `holdout_tasks []` ($\leftarrow$ test/), the data-structure enforcement of P2.

4.2 Task anatomy: query / requirements / trajectory requirements

Each task reconstructs a real interaction where the user *first asks vaguely, then gradually clarifies, and silently inspects the process*. It has three parts:

1. **query** — a colloquial, deliberately under-specified first instruction, carrying only entry info essential to start (which `qN_materials` to use; where to save the deliverable, `result/result_qN`). It simulates a user who has not yet worked out the details.
2. **requirements (content_reqs)** — the true acceptance checklist the user reveals after seeing an unsatisfactory draft: content, format, fields, organization, business rules, personalized preferences. Each item must be **independently verifiable** and at the **same level** (no nested sub-requirements); a scene carries ≥ 12 items. This is the main signal for both the judge and the agent’s evolution.
3. **trajectory requirements (trajectory_reqs)** — constrains/inspects the execution path (tool-call validity and efficiency, allowed information sources, allowed tools/skills; catching hallucination such as fabricating instead of searching). Trajectory requirements are derived *only* from the requirements and must not contradict them.

Content and trajectory are **scored on two separate lines**. At run time only the query reaches the agent; the checklist stays hidden and is disclosed one top-unmet item at a time by the judge (§3.4).

Concrete example (travel scene, warmup Q1, translated): the *query* is one sentence — “Plan a same-day round-trip family itinerary from Hangzhou to Fuyang on 2026-06-01, two adults + one child, no

budget limit; save the plan to `result/result_q1`.” The 12 *requirements* are the hidden checklist (local weather in a basics module; round-trip transport in a transport module; itinerary split by date and morning/afternoon/evening; child-friendly items flagged; walking distance shown separately and ≤ 3 km/day; all facts date-verified with sources shown; ...). The 4 *trajectory requirements* constrain the path (facts must come from authoritative web search, not fabricated; search destination content before planning; save to the specified folder).

4.3 The 75% / 25% train-test overlap

For the test set we deliberately require $\sim 75\%$ of requirements to match the train set and $\sim 25\%$ to be *variants of* or *contradictions to* train requirements. This ratio is not arbitrary: the highly-overlapping part checks whether the agent applies distilled experience **in the right place**; the 25% variant/contradiction part acts as a mirror that exposes (i) over-fitting to the *literal* train requirements, (ii) mistaking a temporary preference for a universal rule, and (iii) failure to override old experience under a new constraint.

Example: a train requirement “on trips, visit more cultural/historical sites” has, in the test set, a **variant** “when planning routes, add more scenic natural spots” and a **contradiction** “do not include any cultural/historical site visits — I don’t like them.” Non-generalizable “noise” requirements are strictly confined to appearing sparingly in the **train** set only, so that pseudo-experience is not retroactively rationalized by the test set — preserving the purity of the evolution-ability measurement.

4.4 Rubric design and grading

Requirements are engineered for **verifiability, not product quality**, to minimize the judge’s randomness on the attempt-count metric. Deterministic requirements are phrased for rule-checkable judgment (fixed headings, fixed CSV columns, fixed metric formulas, numeric thresholds); open-ended parts are graded by the LLM judge against the checklist. The judge is pinned (temperature 0, fixed model, versioned rubric). Cross-runtime neutrality is required: a query must not bind to a specific agent’s tool naming, to avoid gifting points to any one runtime.

4.5 Suite statistics

The released suite has **14 scenes**, each with **4 warmup + 2 holdout = 6 tasks**, totaling **56 warmup + 28 holdout = 84 tasks**, thematically spanning:

Table 8: The 14 benchmark scenes grouped by thematic category.

Category	Scenes
Travel & trip planning	Travel Planning, Business Travel
Shopping / consumer & finance decisions	Healthy-Snack Guide, Cat-Food Guide, Stock Investment Decisions
Data analysis	data_analysis, Sales-Ops Weekly Analytics
Content creation & document/deck authoring	Xiaohongshu Product Ops, PPT Creation, Startup BP / Investor Roadshow
Workplace reporting & ops planning	Weekly-Report Auto-Generation, Team-Building Planning
Information retrieval & research	information_search_gathering
Learning / education	English-Grammar Learning Guide

A minimal smoke suite `assets/benchmarks_demo/hello.json` (1 warmup “Reply with a greeting” + 1 holdout “Introduce yourself”) is shipped for framework-level regression testing. Full benchmarks are compiled from external markdown sources via `python -m src.cli.preprocess`.

4.6 The minimal contract (for third-party benchmarks)

Any benchmark plugged into LIFT must satisfy: (1) **two-phase schema** — explicit `warmup_tasks[]` / `holdout_tasks[]`, each task carrying `query` / `requirements` / `expected_result.{content_reqs, trajectory_reqs}`; (2) **judge-friendly** — `content_reqs` must be a

machine-adjudicable checklist (rules for deterministic parts, LLM rubric for open parts); (3) **cross-runtime neutral** — no tool-name binding; (4) **scenario diversity**.

5 Implementation

Table 9: Key implementation files and their one-line responsibilities.

Topic	Key file	One-liner
Pipeline orchestration	<code>src/lift/pipeline/lift_pipeline.py</code>	<code>repeat × suite × phase</code> multi-level concurrency
Adapter contract (base)	<code>src/lift/adapters/base.py</code>	abstract <code>AgentRuntimeAdapter</code> : <code>worker_judger_factory / evolve_after_warmup</code> hooks
Container orchestration	<code>src/lift/adapters/container_adapter.py</code>	<code>ContainerAgentRuntimeAdapter</code> : <code>resolve_docker_image / start_container + default docker commit</code>
Per-task kernel	<code>src/lift/eval/run_task.py</code>	work + judge review loop
Container policies	<code>src/lift/policies/container.py</code>	warmup / holdout / phase orchestration enums
Status visualization	<code>src/lift/status/</code>	TUI (rich) + HTTP dashboard
Trace backfill	<code>src/postprocess/trace_backfill.py</code>	<code>session_id</code> correlation (calls <code>stitch_phase_langfuse_traces</code>)
Data models	<code>src/models.py</code>	<code>Suite / EvalReport / PhaseRun / Langfuse</code> trace schema
Image build	<code>agent-runtimes/openclaw/build-image.sh</code>	base / with-evolve dual artifacts

Token accounting. LIFT enforces a fixed 5-field token schema across all runtimes: `input_tokens` (excl. cache), `cache_write_tokens`, `cache_read_tokens`, `output_tokens` (incl. `reasoning_tokens` as a subset), `reasoning_tokens` ($\subset$ output). `total_tokens = input + cache_write + cache_read + output` — reasoning is not added again. Evolution cost (`evolve_after_warmup_tokens`) is booked as *training* cost, not Loaded inference cost, so EvolutionROI is not under-counted.

6 Experiments

6.1 Setup and comparison methodology

Runs. We report ten full runs on the EALE benchmark, organized into four base-runtime families so that every cross-runtime observation stays inside a controlled comparison:

- **OpenClaw family** — `openclaw` (implicit), `openclaw_with_openspace` (OpenSpace skill hub), `openclaw_with_agentmemory` (agentmemory backend);
- **Hermes family** — `hermes` (implicit review), `hermes_with_openspace`, `hermes_with_agentmemory`;
- **GenericAgent family** — `genericagent` (implicit file-I/O), `genericagent_active_evolve` (extra active-reflection pass);
- **EvoScientist family** — `evoscientist` (passive baseline: always-on internal memory synthesis, no LIFT-triggered AutoSkills distillation), `evoscientist_active_evolve` (adds an explicit post-hoc AutoSkills pass after the warmup batch that distills observations into skill proposals before commit).

Each run covers $14 \text{ scenes} \times 2 \text{ holdout tasks} \times \text{--repeat } 10$, yielding ≈ 280 (Base, Loaded) pairs per runtime. The agent model, work/judge endpoints, `max_conversation_turns=36`, and container resource passthrough are held identical across all cells; the only manipulated axis is Base vs. Loaded. This grouping lets us read three orthogonal contrasts off the tables: (a) *evolution-strategy* — implicit accumulation vs. an explicit plugin/active pass within one base; (b) *plugin* — OpenSpace vs. agentmemory on the *same* base; and (c) *base agent* — the raw capability floor of OpenClaw vs.

Hermes vs. GenericAgent vs. EvoScientist, read only through delta magnitudes, never absolute scores.

How we compare across ten heterogeneous runtimes. A naïve leaderboard on absolute Loaded scores conflates (a) the underlying runtime’s raw capability, (b) its warmup accumulation policy, and (c) its augmentation plugin — three confounds that make head-to-head rankings scientifically empty. LIFT deliberately restricts cross-runtime inference to *delta magnitudes*, and we further partition the analysis into three tables that respect three distinct comparison boundaries:

1. **Within-runtime (RQ1, Table 10).** Each row is a runtime compared *against its own Base*. This is the only within-row causal claim in the paper; every other comparison is derived from it.
2. **Delta-of-delta within a base family (RQ2, Table 11).** To ask “does an explicit augmentation add value on top of implicit evolution?”, we hold the base runtime fixed and subtract two Base-vs-Loaded deltas: $\text{Gain} = \Delta_{\text{augmented}} - \Delta_{\text{base}}$. Because the OpenClaw and Hermes families each now carry *two* augmentations (OpenSpace and agentmemory), this table doubles as a plugin-vs-plugin contrast on a shared base; the GenericAgent family contributes the implicit-vs-active-reflection contrast. This cancels the raw-capability confound of the underlying LLM/runtime because both sides share the same Base column.
3. **Per-repeat stability (RQ3, Table 12).** We report the mean, std, and 95% CI of $\Delta\text{Turns\%}$ over 10 repeats to expose whether a delta is a robust signal or a single-repeat artifact.

We do **not** rank runtimes by their *absolute* Loaded scores, and we never *subtract* two rows from different base families (e.g. `openclaw+openspace` minus `genericagent` is meaningless). We *do*, however, rank runtimes by their evolution-impact Δ (RQ5, Table 14): since each Δ has already differenced out its own Base, ordering the Δ s answers “who improved most from evolution” without ever comparing raw capability.

Metrics. Let $\bar{B} / \bar{E}$ denote the population means of a metric across all holdout task-repeats. We report the *population-ratio* delta $\Delta X\% = (\bar{E} - \bar{B}) / \bar{B} \cdot 100$ for turns, tool calls, tokens, and latency. Negative means the evolved side is more efficient. Population ratio is preferred over the mean of per-task ratios because the latter is dominated by tasks with near-zero baselines (a single Base = 2, Loaded = 6 task contributes +200%), which flips the direction of the aggregate on Hermes and inflates it on GenericAgent. Pass rates are macro-averaged from the postprocess ALL row of `summary_metrics.csv`.

Pass rate is saturated on EALE. Across the eleven runs, Base and Loaded pass rate are 0.99–1.00 for every runtime except `prime-agent+active`, whose lower band is a run-condition artifact of that sweep discussed in §6.2 (Table 10). Pass rate is therefore a low-resolution signal of self-evolution on this benchmark; the primary lens is interaction efficiency, with pass rate reported for transparency and to flag any regression.

6.2 RQ1 — Do evolved artifacts help on unseen tasks?

Each row of Table 10 is a within-runtime Base-vs-Loaded comparison.

Reading Table 10. `hermes` and both its augmented siblings show the strongest interaction-cost reduction on every axis (turns -16.6 to -19.1% , latency ≈ -22 to -23% , tokens ≈ -17 to -19%); its review-driven implicit evolution distills verbose warmup into a compact plan that the model consults on Loaded runs. The `genericagent` family moves only marginally ($|\Delta| \leq 3\%$ on all axes, in either direction) — its file-I/O policy captures skill files but the base runtime already writes highly focused deliverables, leaving little conversational cost to cut; the extra active-reflection pass does not change this and even drifts slightly inefficient ($+1.9\%$ turns). The `openclaw` family is the split case: the base and its `agentmemory` variant hover near zero or slightly regress on turns/tokens/latency ($+0.75$ to $+3.3\%$), whereas `OpenSpace` is the only `OpenClaw` configuration that turns clearly efficient (-4.0% turns, -7.1% tokens) — natural memory accumulation without an explicit distillation hook produces state that lengthens rather than shortens the outer loop, and swapping in an `agentmemory` backend does not by itself fix this. `evoscientist` is the standout non-Hermes row: it cuts turns -14.3% , tokens -10.2% , and latency -15.1% , and posts the single largest tool-call reduction in the paper (-34.7%) — its always-on internal memory loop writes synthesized profiles during warmup (not just raw observations), and its verbose Base (186 tool calls/task, Table 13) leaves

Table 10: **RQ1: Base vs. Loaded across the 14 scenes** (holdout only, macro-averaged over 10 repeats). Negative $\Delta\text{Turns}\%$ / $\Delta\text{Tools}\%$ / $\Delta\text{Tokens}\%$ / $\Delta\text{Latency}\%$ mean the evolved side is *more efficient*. Tasks column: total (post-processed after excluding cells with missing traces).

Runtime	Tasks	Base Pass	Loaded Pass	ΔPass	$\Delta\text{Turns}\%$	$\Delta\text{Tools}\%$	$\Delta\text{Tokens}\%$	$\Delta\text{Latency}\%$
openclaw	280 (269)	0.9929	0.9929	+0.00	+2.74	-2.09	+0.44	+3.34
openclaw+openspace	280 (271)	1.0000	1.0000	+0.00	-3.95	-6.41	-7.12	-7.90
openclaw+agentmemory	278 (265)	0.9964	0.9964	+0.00	+0.75	-1.19	+2.28	-0.18
hermes	278 (272)	0.9964	0.9928	-0.36	-17.24	-10.38	-17.48	-23.01
hermes+openspace	280 (268)	0.9964	1.0000	+0.36	-19.09	-10.62	-19.27	-22.09
hermes+agentmemory	280 (269)	0.9964	0.9964	+0.00	-16.64	-8.82	-17.34	-22.97
genericagent	275 (257)	0.9964	1.0000	+0.36	-1.75	-1.38	-0.53	-2.20
genericagent+active	274 (263)	1.0000	1.0000	+0.00	+1.92	+0.92	+1.68	+3.17
evoscientist	280 (248)	1.0000	0.9929	-0.71	-14.33	-34.65	-10.23	-15.05
evoscientist+active	278 (250)	1.0000	1.0000	+0.00	-14.69	-27.57	-12.73	-16.57
prime-agent+active	280 (260)	0.9321	0.9429	+1.07	-3.27	+5.66	+3.21	-5.73

the most compressible slack of any runtime. Its active-AutoSkills sibling *evoscientist+active* lands almost on top of it on turns (-14.7%), latency (-16.6%), and tokens (-12.7%), but its tool-call cut is *smaller* (-27.6% vs. -34.7%) — the explicit post-hoc distillation does not add efficiency over the always-on loop it sits on and slightly erodes the tool-call advantage (analysed in RQ2/RQ3). Pass rate is essentially unchanged across the first ten runs ($|\Delta\text{Pass}| \leq 0.71\text{pp}$, the passive *evoscientist* row’s two dropped Loaded successes out of 280; the active variant holds 1.0000 on both sides), consistent with a saturated benchmark. The one row that sits off the saturation band is *prime-agent+active* (Base 0.9321, Loaded 0.9429, $\Delta\text{Pass} +1.07\text{pp}$): its lower absolute pass rate is a run-condition artifact of that sweep — baseline tasks lost to per-turn timeout / session-lock hangs, not benchmark difficulty — and both phases ran under the same timeout regime, so its within-runtime Δ s remain a valid Base-vs-Loaded contrast (it posts the run’s only positive ΔPass).

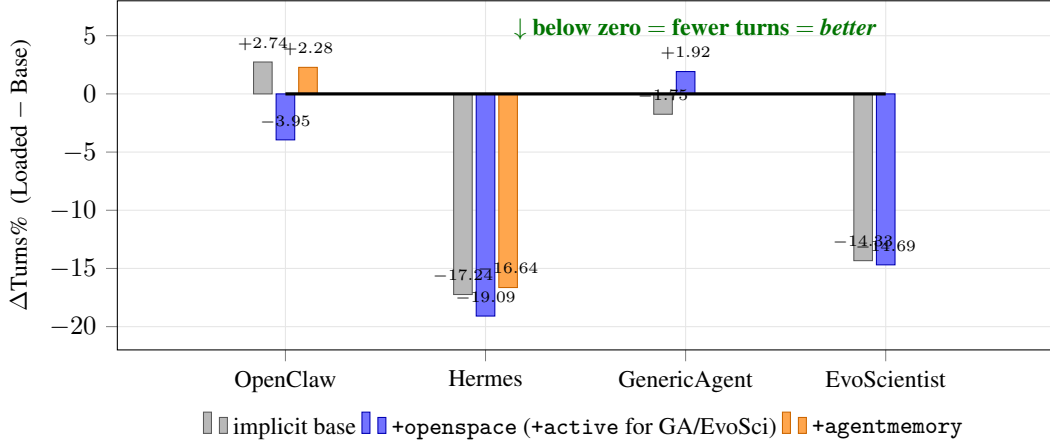


Figure 1: **RQ1 at a glance: $\Delta\text{Turns}\%$ by base family** (from Table 10; GenericAgent and EvoScientist have no *plugin* variants but do have an *active*-evolution variant, plotted in the second series — so their *+agentmemory* bar is blank). Bars *below* the thick zero line are efficiency *gains* (evolved side uses fewer turns — **better**); bars *above* zero are regressions. Note that $\Delta\text{Turns}/\text{Tools}/\text{Tokens}/\text{Latency}$ are “lower-is-better” whereas ΔPass (not shown, saturated) is “higher-is-better”. The four grey bars are the pure implicit-evolution effect of each base runtime and are directly comparable as an *evolution-impact* ranking (Hermes $\gg$ EvoScientist $\gg$ GenericAgent $\gtrsim$ OpenClaw); see §6.6.

6.3 RQ2 — Do augmentation plugins add value on top of implicit evolution?

Every augmented runtime is compared against the *implicit* base of its own family. The Gain column is a *delta-of-delta*: $\text{Gain} = \Delta_{\text{augmented}} - \Delta_{\text{base}}$, in the same units as the underlying Δ . A *negative* Gain

means the augmentation improves the metric *beyond* implicit evolution alone — i.e., the explicit mechanism adds real value on top of natural state accumulation. Reading the table three ways answers three distinct questions: comparing two rows that share a base (e.g. the two `openclaw` rows) is a **plugin-vs-plugin** contrast (OpenSpace vs. agentmemory); comparing the base row’s implicit Δ against its augmented rows is an **evolution-strategy** contrast (implicit vs. explicit / active); and comparing Gain magnitudes *across* families is a **base-agent** contrast, admissible here only because the delta-of-delta has already cancelled each base’s raw-capability floor.

Table 11: **RQ2: augmentation gain (delta-of-delta, same base runtime).** Each row subtracts the implicit base Δ of its own family from the augmented Δ . Negative Gain = augmentation is more efficient than the implicit-only baseline. Rows sharing a base (OpenClaw, Hermes) form a plugin-vs-plugin contrast; the GenericAgent and EvoScientist rows are implicit vs. active (reflection / AutoSkills distillation).

Augmentation (base)	Turns			Tool calls			Tokens		
	Δ Base	Δ Aug	Gain	Δ Base	Δ Aug	Gain	Δ Base	Δ Aug	Gain
<code>openspace</code> (<code>openclaw</code>)	+2.74	−3.95	−6.69	−2.09	−6.41	−4.32	+0.44	−7.12	−7.55
<code>agentmemory</code> (<code>openclaw</code>)	+2.74	+0.75	−1.99	−2.09	−1.19	+0.90	+0.44	+2.28	+1.84
<code>openspace</code> (<code>hermes</code>)	−17.24	−19.09	−1.85	−10.38	−10.62	−0.24	−17.48	−19.27	−1.79
<code>agentmemory</code> (<code>hermes</code>)	−17.24	−16.64	+0.60	−10.38	−8.82	+1.56	−17.48	−17.34	+0.14
<code>active</code> (<code>genericagent</code>)	−1.75	+1.92	+3.67	−1.38	+0.92	+2.30	−0.53	+1.68	+2.21
<code>active</code> (<code>evoscientist</code>)	−14.33	−14.69	−0.36	−34.65	−27.57	+7.08	−10.23	−12.73	−2.50

Reading Table 11. The three contrasts tell a coherent story about *where* an augmentation helps:

- **Plugin vs. plugin (OpenSpace $\gg$ agentmemory on both shared bases).** On `openclaw`, OpenSpace delivers the largest gain in the paper (−6.7pp turns, −7.6pp tokens) while agentmemory yields only a modest turns gain (−2.0pp) and is slightly *negative* on tokens (+1.8pp). On `hermes` the ordering repeats: OpenSpace is marginally positive (−1.9pp turns) whereas agentmemory is essentially neutral-to-negative (turns +0.6pp, tools +1.6pp). A structured skill hub compresses the outer loop more reliably than a raw memory backend, which stores observations without a distillation step that shortens future interactions.
- **Evolution strategy (explicit / active is not free).** The GenericAgent active-reflection pass *costs* rather than saves (+3.7pp turns, +2.2pp tokens): the extra reflection turns are not amortized because the base already writes focused deliverables, so the added deliberation is pure overhead on this benchmark. The EvoScientist active-AutoSkills pass tells a subtler cautionary tale: on a base whose implicit loop *already* evolves strongly (−14.3% turns), the explicit post-hoc distillation returns essentially *zero marginal gain* on the primary axis (turns Gain −0.36pp, within run-to-run noise) and is measurably *worse* on tool calls (Gain +7.08pp: active cuts tools only −27.6% versus the passive base’s −34.7%). It does buy a small tokens improvement (−2.5pp) and, more interestingly, a tighter per-repeat spread (RQ3 std 5.77 versus 9.52), so the AutoSkills pass mainly *stabilises* an already-strong signal rather than enlarging it — while adding real warmup cost (the distillation graph fires an extra LLM pass per suite). The reading matches GenericAgent’s: an explicit active step is only worth its overhead when the implicit base has left compressible slack *on the table*; layered on a base that already self-distills, it is at best a variance-reduction knob, not an efficiency lever.
- **Base agent (headroom is set by the base’s warmup policy).** The same OpenSpace plugin gives −6.7pp on OpenClaw but only −1.9pp on Hermes — a 3–4 $\times$ gap — because Hermes’s review step has already extracted most of the compressible cost, leaving little for the plugin to sharpen. Augmentation value is therefore inseparable from the host base: a plugin’s absolute effect cannot be quoted without naming its runtime.

6.4 RQ3 — Is the delta stable across repeats?

Table 12 reports per-repeat statistics: for each of the 10 repeats we compute a population Δ (as in Table 10), then report mean, std, and 95% CI over the 10 per-repeat values. $CI = \text{mean} \pm 1.96 \cdot \text{std} / \sqrt{10}$.

Table 12: **RQ3: Per-repeat stability across the 10 repeats.** Std is over per-repeat population Δ s. Five rows — the three Hermes configurations and *both* EvoScientist variants — have turns-CIs sitting fully below zero, giving the strongest statistical claim; every OpenClaw and GenericAgent CI crosses zero on turns, so those small Δ s are noise-dominated at 10-repeat resolution.

Runtime	Δ Turns% mean	std	95% CI	Δ Tools% mean	std
openclaw	+3.13	9.40	[−2.70, +8.95]	−1.84	8.28
openclaw+openspace	−3.78	7.22	[−8.26, +0.69]	−6.17	7.62
openclaw+agentmemory	+1.21	8.47	[−4.04, +6.47]	−1.03	7.69
hermes	−17.06	7.58	[−21.76, −12.36]	−9.83	11.74
hermes+openspace	−18.89	7.56	[−23.58, −14.20]	−10.00	10.60
hermes+agentmemory	−16.57	6.31	[−20.48, −12.66]	−7.95	15.71
genericagent	−1.40	7.91	[−6.30, +3.50]	−0.97	9.30
genericagent+active	+2.11	6.63	[−2.00, +6.22]	+0.92	8.29
evoscientist	−13.69	9.52	[−19.59, −7.79]	−27.89	22.34
evoscientist+active	−14.60	5.77	[−18.18, −11.02]	−27.11	39.77
prime-agent+active	−3.05	6.31	[−6.96, +0.87]	+6.21	10.67

All three Hermes rows deliver a robust turns-reduction signal at 10-repeat resolution (every CI well below zero), including hermes+agentmemory whose tighter std (6.31) gives the narrowest interval [−20.48, −12.66]. evoscientist joins them as the fourth repeat-stable row: despite the widest std of the significant group (9.52), its large mean gain keeps the whole interval [−19.59, −7.79] below zero — a robust efficiency signal from a runtime with *no* per-task distillation hook (Appendix C), showing that distillation is sufficient but not necessary for a repeat-stable gain. Its active sibling evoscientist+active is the fifth such row and, tellingly, its interval is the *tightest* of the two EvoScientist runs ([−18.18, −11.02], std 5.77 vs. 9.52) at a near-identical mean (−14.60 vs. −13.69): the explicit post-hoc AutoSkills pass roughly *halves* the per-repeat variance without moving the point estimate, i.e. it makes an already-strong gain more *reliable* rather than larger — the only clear benefit we can attribute to the active step. openclaw+openspace’s CI [−8.3, +0.7] narrowly straddles zero: the Gain in Table 11 is real in aggregate but only marginally repeat-stable. Every remaining OpenClaw and GenericAgent CI crosses zero on both turns and tools — including the two new rows (openclaw+agentmemory [−4.04, +6.47], genericagent+active [−2.00, +6.22]) — so we treat their small point estimates as directional only, not statistically significant. The updated picture: the Hermes family and evoscientist carry a strong repeat-stable signal, OpenClaw+OpenSpace a marginal one, and agentmemory / active reflection none.

6.5 RQ4 — Cost view: tokens and latency per task

The Base-turns column is remarkably flat across runtimes (13.5–16.9), so the Δ s translate almost directly to absolute turn-count savings: the Hermes family saves ≈ 2.8 – 3.1 turns per task, OpenClaw+OpenSpace ≈ 0.6 , and the OpenClaw+agentmemory / GenericAgent variants land within ± 0.3 turns of their base. Latency savings track turns closely on the Hermes family (all around −22 to −23%) but decouple on OpenClaw variants where OpenSpace saves latency (−7.9%) faster than it saves turns (−4.0%) — consistent with the skill hub replacing sequential tool-call chains by a single retrieved skill invocation — while OpenClaw+agentmemory shows the opposite mild pattern (latency −0.2% vs. tokens +2.3%), a memory backend that neither compresses nor bloats the loop. These Loaded columns exclude evolve_after_warmup tokens (booked as training cost, §5), so EvolutionROI derivations do not double-count.

Table 13: **RQ4: Per-task means with token totals.** Base turns / tools / tokens are absolute per-task population means; Δ columns repeat Table 10 for direct cost comparison. Tokens are the full 5-field sum (input + cache_write + cache_read + output) from Langfuse trace_backfill.

Runtime	Base turns	Base tools	Base tokens	Δ Turns%	Δ Tools%	Δ Tokens%	Δ Latency%
openclaw	15.9	33.2	2 139 707	+2.74	-2.09	+0.44	+3.34
openclaw+openspace	16.0	34.0	2 442 614	-3.95	-6.41	-7.12	-7.90
openclaw+agentmemory	15.7	32.2	2 427 722	+0.75	-1.19	+2.28	-0.18
hermes	16.9	39.0	2 623 025	-17.24	-10.38	-17.48	-23.01
hermes+openspace	16.3	39.5	2 888 667	-19.09	-10.62	-19.27	-22.09
hermes+agentmemory	16.6	40.5	3 076 998	-16.64	-8.82	-17.34	-22.97
genericagent	13.9	68.5	2 944 209	-1.75	-1.38	-0.53	-2.20
genericagent+active	13.5	62.1	1 630 682	+1.92	+0.92	+1.68	+3.17
evoscientist	15.6	186.1	7 275 312	-14.33	-34.65	-10.23	-15.05
evoscientist+active	15.7	199.9	7 181 170	-14.69	-27.57	-12.73	-16.57
prime-agent+active	16.6	10.7	2 170 430	-3.27	+5.66	+3.21	-5.73

6.6 RQ5 — Ranking runtimes by evolution impact

Table 10 deliberately reports each runtime *against its own Base*, and §6.1 forbids ranking runtimes by their *absolute* Loaded scores (that would conflate raw capability, warmup policy, and plugin). It does *not*, however, forbid ranking by the quantity LIFT is designed to measure — the **evolution impact** Δ itself. Because every Δ is a within-runtime Base-vs-Loaded contrast, the raw-capability floor is already differenced out of each number, so ordering runtimes by Δ is a legitimate “who improved most from evolution” ranking (distinct from “who is best out of the box”, which we never claim). Table 14 sorts all eleven runtimes by Δ Turns% (primary efficiency axis; more negative = larger evolution gain = better rank).

Table 14: **RQ5: cross-runtime evolution-impact ranking**, sorted by Δ Turns% (most negative first). This ranks the *size of the improvement each runtime obtains from evolution*, not its absolute capability. Sig.: does the RQ3 95% CI for Δ Turns% lie entirely below zero? ✓ = yes (robust), - = CI crosses zero (directional only).

Rank	Runtime	Δ Turns%	Δ Tools%	Δ Tokens%	Δ Latency%	Sig. (turns)
1	hermes+openspace	-19.09	-10.62	-19.27	-22.09	✓
2	hermes	-17.24	-10.38	-17.48	-23.01	✓
3	hermes+agentmemory	-16.64	-8.82	-17.34	-22.97	✓
4	evoscientist+active	-14.69	-27.57	-12.73	-16.57	✓
5	evoscientist	-14.33	-34.65	-10.23	-15.05	✓
6	openclaw+openspace	-3.95	-6.41	-7.12	-7.90	-
7	prime-agent+active	-3.27	+5.66	+3.21	-5.73	-
8	genericagent	-1.75	-1.38	-0.53	-2.20	-
9	openclaw+agentmemory	+0.75	-1.19	+2.28	-0.18	-
10	genericagent+active	+1.92	+0.92	+1.68	+3.17	-
11	openclaw	+2.74	-2.09	+0.44	+3.34	-

Reading Table 14. The top of the board is dominated by a *base-runtime* effect, not a plugin effect: the entire Hermes family (ranks 1–3) clusters tightly, and no plugin moves a runtime within it across more than ~ 2.5 pp. Within the Hermes cluster the three configurations are statistically indistinguishable (Δ Turns% spread < 2.5 pp, all CIs overlapping), so ranks 1–3 should be read as “Hermes’ review distillation”, not “OpenSpace won”. The two EvoScientist rows (ranks 4–5) are the important addition: they sit only ≈ 2 – 2.5 pp behind the Hermes cluster and are the *fourth and fifth* repeat-stable rows (RQ3), so the sharp gap that earlier separated Hermes from everything else is now bridged by a non-Hermes family. Crucially the passive evoscientist (rank 5, *no* per-task distillation) and its active AutoSkills variant (rank 4) are statistically indistinguishable on turns (Δ spread 0.36pp, overlapping CIs), and the active pass is actually *weaker* on tool calls (-27.6% vs. -34.7%): the explicit distillation reorders them by a hair but buys no separable efficiency gain over the always-on internal loop it sits on. There is instead a clear break *below* rank 5: ranks 6–10 cluster

within $\pm 4\text{pp}$ of zero and are *not* statistically separable from each other or from “no effect” at 10 repeats — we print their order for completeness but caution against reading ranks 6 vs. 7 vs. 8 as a real difference. **One caveat governs the whole table:** a large Δ partly reflects how much *compressible slack* the Base run carried — Hermes’ and EvoScientist’s verbose baselines leave more to cut than GenericAgent’s already-terse one (EvoScientist’s 186 tool-calls/task Base is the most verbose we measured) — so this is a ranking of *realized evolution gain on EALE*, not of the intrinsic quality of each evolution mechanism. Appendix C (Table 16) spells out exactly what each runtime does to evolve — which hook fires, when, and what it invokes — and shows the five repeat-stable rows do *not* share a single mechanism class: three are per-task distillation (Hermes), one is carry-only with an always-on internal memory loop (evoscientist), and one adds an explicit post-hoc AutoSkills reflection on top of that same loop (evoscientist+active), so distillation is *sufficient but not necessary* for the robust signal above — and, as the two EvoScientist rows show, an *explicit* distillation step on a base that already self-synthesises is neither necessary nor individually beneficial beyond variance reduction.

6.7 Findings and negative results

We summarize four robust findings and one deliberate non-finding:

1. **Implicit evolution’s efficacy depends heavily on the runtime’s warmup policy — and more than one policy works.** Hermes’ review-driven distillation delivers a strong, repeat-stable interaction-cost reduction (turns -17.2% , latency -23.0% , 95% CI fully below zero), and this holds across all three Hermes configurations. EvoScientist reaches a comparable repeat-stable gain (turns -14.3% , 95% CI fully below zero) by a *different* policy: an always-on internal loop that synthesizes structured profiles during warmup rather than distilling per task. OpenClaw’s plain state-accumulation policy, by contrast, actually *regresses* slightly on turns/tokens/latency, indicating that merely accumulating undistilled state is not a reliable win. GenericAgent’s file-I/O policy produces a tiny effect that is not statistically distinguishable from zero at 10 repeats. The dividing line is not distillation-vs-not but whether the warmup policy produces a *compacted, reusable* artifact at all.
2. **Plugin choice matters more than plugin presence: OpenSpace $\gg$ agentmemory.** On the two bases that carry both plugins, a structured skill hub (OpenSpace) consistently out-compresses a raw memory backend (agentmemory). On OpenClaw, OpenSpace *rescues* an otherwise-neutral base (Gain: turns -6.7pp , tokens -7.6pp) whereas agentmemory is neutral-to-negative (turns -2.0pp , tokens $+1.8\text{pp}$); on Hermes, OpenSpace adds a modest -2pp on turns/tokens while agentmemory adds nothing ($+0.6\text{pp}$ turns). The distillation step, not the storage substrate, is what shortens future interactions.
3. **Explicit / active evolution is not automatically better than implicit.** GenericAgent’s extra active-reflection pass *costs* interaction budget rather than saving it (Gain: turns $+3.7\text{pp}$, tokens $+2.2\text{pp}$): on a benchmark where the base already writes focused deliverables, the added deliberation is pure overhead. EvoScientist’s active-AutoSkills pass makes the point from the opposite direction: layered on a base whose implicit loop *already* evolves strongly, the explicit distillation adds *no* separable gain (turns -14.7% vs. -14.3% , statistically tied; tool calls actually *worse*, -27.6% vs. -34.7%) and its one measurable effect is to halve per-repeat variance (RQ3 std 5.77 vs. 9.52) — at the cost of an extra warmup LLM pass. More evolution machinery does not imply more downstream efficiency; at best it buys stability once an implicit mechanism is already saturating the available slack.
4. **Statistical significance is unevenly distributed, and not confined to one mechanism.** Five rows show 95% CIs on $\Delta\text{Turns}\%$ that lie entirely below zero (Table 12): the three Hermes configurations *and both* EvoScientist variants; every OpenClaw and GenericAgent row’s turns-CI crosses zero. Crucially the passive evoscientist performs no per-task distillation, so a repeat-stable double-digit gain is achievable without the review-style hook — distillation is sufficient but not necessary — while its active sibling shows the converse limit: adding an explicit distillation hook on top of an already-evolving base tightens the CI but does not move the mean. We report the point estimates for completeness but recommend against reading small differences ($|\Delta| \lesssim 5\%$) as effects at 10-repeat resolution. Pass rate on EALE is saturated (0.99–1.00 on both Base and Loaded for every runtime except

prime-agent+active, largest $|\Delta\text{Pass}| = 1.07\text{pp}$), so it is retained only as a guardrail against silent regressions — none observed.

5. **We rank evolution impact, not raw capability.** The eleven runtimes share a benchmark but not a raw-capability floor, a warmup policy, or an augmentation surface, so we produce no “best agent” by absolute Loaded score. What we *do* rank (RQ5, Table 14) is the evolution-impact Δ — a within-runtime quantity from which each Base has already been differenced out. On that axis the Hermes family leads ($\Delta\text{Turns}\% \approx -17$ to -19%) with both EvoScientist variants close behind (-14.3 to -14.7%); these five are the repeat-stable rows, the remaining six cluster within $\pm 4\text{pp}$ of zero and are statistically inseparable. The ordering is best read as an *evolution-mechanism* comparison, but not a single-mechanism story: robust gains come both from Hermes’ per-task review distillation and from EvoScientist’s always-on internal memory synthesis, so “distillation” is one sufficient route rather than the only one. The one confound we flag is compressible slack: a verbose Base leaves more room for evolution to cut, so large Δ s are partly a property of the Base, not only of the evolution mechanism — EvoScientist’s very verbose Base (186 tool-calls/task) is a clear instance.

7 Discussion and Limitations

1. **Benchmark data contamination.** LIFT cannot prevent the gold set from being contaminated by LLM training data; ABC-Checklist-style tools [Zhu et al., 2025] must gate this at benchmark-design time.
2. **No direct cross-agent comparison on absolute scores.** We deliberately do **not** support head-to-head *OpenClaw+plugin* vs. *Hermes* comparison on raw Loaded performance — too many confounds make such conclusions non-attributable. The sanctioned move is per-agent Base vs. Loaded, then compare the *delta magnitudes* (the evolution-impact ranking of RQ5, §6.6), which is confound-free because each Δ is measured against its own Base.
3. **Evolution $\neq$ artifacts.** LIFT is artifact-source-agnostic by default. To isolate “the evolution *mechanism* has value,” use the attribution triplet (No-Evo / Only-Products / Evo-On) — an optional appendix, not a required metric.
4. **Judge bias.** When both work and judge are LLMs, self-preference is a risk. Current mitigation: the judge role-plays “the user,” adjudicates the `content_reqs` checklist item-by-item into a single completion ratio, and is pinned (temperature 0, fixed model, versioned rubric); work and judge are additionally split into sibling containers (§3.4) so self-preference cannot compound with state leakage — neither party can read or write the other’s runtime state. Making deterministic requirements fully machine-checkable — shrinking the subjective surface further — is future work.
5. **Environment fidelity.** The container is still a *controlled Linux environment*; fidelity gaps remain versus desktop-GUI / browser agents — future work.
6. **Dimensions the relation matrix (Appendix B) does not cover.** Three axes were considered and deliberately left out. *Cost per artifact-unit* (dollars-per-percent-lift) requires a cost model of the evolution mechanism, which is runtime-internal and out of LIFT’s scope. *Multi-agent / group memory* is partially covered by the framework’s group-memory mixin, but no benchmark yet forces multi-agent Base-vs-Loaded — an explicit v2 direction. *Human-in-the-loop grading* is out of scope by construction: LIFT’s contribution is a mechanistic protocol runnable unattended by a third party; HITL is orthogonal and can be layered on top for high-stakes deployment.

8 Conclusion

Our stance in one sentence: **evaluating a self-evolving agent is not about how much better it gets on tasks it has already seen, but how much better it gets on *unseen* tasks by virtue of its distilled artifacts.** LIFT realizes this with train/test separation + same-task paired runs + container snapshots + a three-layer abstraction + multi-level concurrency + trace backfill, and ships a self-contained 14-scene / 84-task benchmark. We hope to provide a reusable, criticizable, comparable base for evaluation infrastructure.

Reproducibility & Release

- **Code:** framework, adapters for mainstream self-evolving agent runtimes, status dashboard, post-processing — released at <https://github.com/FeiZhuNiU-INFJA/LIFT>.
- **Images:** `lift-openclaw-base / lift-openclaw-with-openspace / lift-openclaw-with-agentmemory` and `lift-{genericagent,hermes,hermes-with-openspace,hermes-with-agentmemory,openhuman,openhuman-with-agentmemory,evoscientist}:latest` build scripts.
- **Data:** 14-scene benchmark (EALE) hosted at <https://huggingface.co/datasets/FeiZhuNiU-INFJA/EALE> under **CC BY 4.0**, with a dataset card and Croissant metadata (Appendix C summarizes the datasheet); plus the demo smoke suite in-repo. EALE is file-structured (per-task folders + materials), so it is consumed by cloning the repository tree rather than via a single-table `load_dataset`.
- **Entry points:** `python-msrc.cli.lift_main-r<runtime>--benchmark_dir<dir>--suite<name>.json--run_id<id>; -evaluate-only` for post-process replay.

A Surveyed open-source works (by type)

We group the works surveyed along the axis “can it answer *do the artifacts help?*”. Methods / judge: ABC Checklist [Zhu et al., 2025], Agent-as-a-Judge [Zhuge et al., 2024], MAJ-Eval [Chen et al., 2026], AgentDistill [Qiu et al., 2025]. Evolution / artifact benchmarks: SEA-Eval [Jiang et al., 2026], EvolveTool-Bench [Kaliyev and Maryanskyy, 2026], SkillsBench [Li et al., 2026], EvoMem-Bench [Wang et al., 2026], Evo-Memory [Wei et al., 2025]. Lifelong / sequential: LifelongAgentBench [Zheng et al., 2025]. General task benchmarks: REAL [Garg et al., 2025], MLR-Bench [Chen et al., 2025], AgentIF [Qi et al., 2025], τ -bench / τ^2 -bench [Yao et al., 2024, Barres et al., 2025], AgentBench [Liu et al., 2023], GAIA [Mialon et al., 2023], SWE-bench [Jimenez et al., 2023], and harness-comparison benchmarks. Safety: OS-Harm [Kuntz et al., 2025], RAS-Eval [Fu et al., 2025], MultiBreak [Song et al., 2026]. Evaluation SDKs: DeepEval [Confident AI, 2025], Opik [Comet ML, 2025], Promptfoo [Promptfoo, 2025], with Langfuse [Langfuse, 2025] used for tracing.

B LIFT vs. existing work (relation matrix)

Table 15: Relation matrix contrasting LIFT’s stance with existing work along nine dimensions.

Dimension	LIFT’s stance	Direct contrast
Artifact source	source-agnostic; test whether the artifact itself helps	SkillsBench empirically shows self-generated skills often useless; LIFT provides the infrastructure to test this
Causal contrast	Base vs. Loaded, same-task paired	most benchmarks lack this control group
Train / test	explicit separation (warmup/holdout)	most self-evolution papers mix them
Per-task scoring	work + judge review loop	borrowes Agent-as-a-Judge tool-augmented verification
Artifact materialization	<code>docker commit</code> $\rightarrow$ delta image	more reproducible than toggle-load, more complete than structured export
Learning curves	evolution-specific diagnostic (optional)	LifelongAgentBench makes FWT/BWT primary; LIFT makes them diagnostic
Cross-agent comparison	per-agent Base-vs-Loaded, compare deltas; no head-to-head	harness-comparison benchmarks compare bare agents
Safety	cross-cutting, optional	OS-Harm / RAS-Eval provide dedicated safety benchmarks; LIFT does not redo them
Trace	Langfuse backfill	like Opik, but trace-only, no evaluator

C Evolution mechanisms by runtime

Because RQ5 (Table 14) reads the ranking as an *evolution-mechanism* comparison rather than a capability leaderboard, we spell out here exactly what each runtime does to evolve — which hook it

overrides, when the hook fires, what it invokes, and what state the `delta docker commit` captures. Every entry is verified against the adapter source under `src/lift/adapters/`. The framework fixes the ordering `evolve_after_task` (after each warmup task) $\rightarrow$ `evolve_after_warmup` (after the whole warmup batch) $\rightarrow$ `docker commit` (freeze delta), so any distillation step always runs *before* the delta is frozen. Note that `docker commit` snapshots the entire writable container layer; the paths listed below are the runtime-declared “true evolution artifacts” we use for the pre-commit `docker diff` audit, not a restriction on what is committed (the `/workspace/task` bind mount is never captured).

Three mechanism classes, and what the ranking does (and does not) follow. The eleven runtimes fall into three families of mechanism. **(C) Carry-only** (`openclaw`, `+openspace`, `+agent-memory`, `genericagent`, and `evoscientist`): there is *no* LIFT-triggered distillation hook; the delta is whatever memory/skill files the agent wrote during warmup, snapshotted by `docker commit`. **(D) Per-task distillation** (the entire Hermes family, plus `prime-agent+active`): after *every* warmup task, before the next one starts, a blocking step compresses that task’s trajectory into a compact reusable artifact — a review into skills/memory for Hermes, a `/refine -global` pass into the global Continual Harness for prime-agent. **(R) Explicit post-hoc reflection** (`genericagent+active` and `evoscientist+active`): an extra post-warmup pass distills the batch — for `GenericAgent`, two additional reflection chats (after each task and after the batch); for `EvoScientist`, one `AutoSkills` maintenance-graph run that turns accumulated observations into explicit skill proposals before commit. Mapping this onto RQ5, per-task distillation **(D)** *as implemented on Hermes* reliably produces a robust, repeat-stable, double-digit gain — it is precisely the step that turns a verbose warmup transcript into a short plan the model consults on Loaded runs. But the class-**(D)** hook is neither sufficient nor necessary on its own. It is *not sufficient*: `prime-agent+active` runs the same per-task-distillation family (a `/refine -global` pass) yet lands at only -3.27% turns (rank 7, CI crosses zero), because its Base is the *tersest* we measured (10.7 tool-calls/task, Table 13) — with almost no compressible slack, even a genuine distillation step has little to cut. And it is *not necessary*: `evoscientist` sits in class **(C)** yet is repeat-stable (-14.3% turns, CI fully below zero, rank 4). The distinction is that its always-on internal loop does not merely accumulate raw observations — it continuously *synthesizes* structured profiles (`SOUL/USER_PROFILE/RESEARCH_TASTE/PROJECT_PROFILE`) during warmup, which function as a compact plan much like a distilled skill even though no explicit distillation hook fires. The remaining class-**(C)** runtimes (`openclaw`, `genericagent`), which carry only undistilled files, stay neutral or mildly negative (accumulated state lengthens context without shortening the outer loop); class **(R)**’s extra deliberation *adds* interaction cost on `GenericAgent`, where the base already writes focused deliverables. `EvoScientist`’s class-**(R)** variant is the more instructive case: bolted onto a base whose class-**(C)** internal loop is *already* repeat-stable, its explicit `AutoSkills` distillation neither enlarges the turns gain (-14.7% vs. the passive -14.3% , statistically tied) nor improves tool efficiency (in fact -27.6% vs. -34.7% , *worse*); its one measurable effect is halving the per-repeat variance (RQ3 std 5.77 vs. 9.52) at the price of an extra warmup LLM pass. The refined reading: what predicts a robust gain is not the presence of a distillation *hook* per se, but whether *some* process — explicit per-task review *or* an always-on synthesis loop — turns the verbose warmup trace into a compact, reusable artifact before the Loaded run; and once such a process exists, layering an *additional* explicit distillation on top yields at most stability, not extra efficiency.

A caveat on the passive agentmemory rows. The two `+agentmemory` configurations reported here run `agentmemory` in its *passive* mode: the local `:3111` server is started without an LLM provider, so its built-in `consolidate/reflect` distillation never fires and only raw observations are persisted. The framework also ships an *active* variant (`openclaw_with_agentmemory_active_evolve`) that, after warmup, explicitly POSTs `/agentmemory/crystals/auto` followed by `/agentmemory/consolidate-pipeline {tier:all}` to distill raw observations into semantic facts and higher-order insights before committing. That active path was not part of this v1 sweep; consequently the near-zero `+agentmemory` deltas should be read as “an undistilled memory store does not by itself help,” *not* as a verdict on `agentmemory`’s distillation engine. Closing that gap (active `agentmemory`, plus the `openclaw_with_evolve_learn_review` path) is an explicit v2 direction. We already close one arm of it here: `EvoScientist`’s *active* `AutoSkills` variant (`evoscientist_active_evolve`) is run on top of the passive `evoscientist` baseline and reported throughout §6, and its finding — that an explicit distillation pass layered on an already-self-synthesising base tightens per-repeat variance but adds no separable mean gain (and slightly erodes

tool-call efficiency) — is exactly the kind of null-with-a-twist result this active/passive contrast was designed to surface.

Why prime-agent is evaluated only in its +active form. Unlike the runtimes above, prime-agent has no meaningful passive counterpart to pair against. Its Continual Harness is *session-local by default*: ordinary task execution writes harness edits bound to that conversation’s session id. Because every LIFT holdout task runs in a fresh container with a fresh session, that session-local state is unreadable at evaluation time, so a purely passive prime-agent (warmup writes carried by `docker commit` alone) transfers ≈ 0 usable artifact — its Loaded runs would, by construction, see essentially nothing. The `/refine -global` pass is precisely what escapes this: it promotes the distilled increment to the session-independent layer (`~/.prime/agent/harness/`) that the commit can carry and a new session can read. A passive prime-agent row would therefore be a near-zero *negative control* rather than a second comparable data point, which is why RQ2 reports no prime-agent pair and §6 lists only prime-agent+active. This makes prime-agent’s own rank a within-runtime statement (does explicit global `/refine` help *this* agent), read against the same ≈ 0 passive floor that its architecture guarantees.

D Datasheet summary (EALE)

Following *Datasheets for Datasets* [Geburu et al., 2021], we summarize EALE here; the full datasheet ships with the dataset release.

- **Motivation.** Measure whether a self-evolving agent genuinely improves on *unseen* tasks after consolidating artifacts from warmup tasks, isolating genuine generalization from pseudo growth (leakage, context pollution, grader self-preference, non-reproducible runtimes).
- **Composition.** 14 scenes $\times$ (4 warmup + 2 holdout) = 84 tasks. Each instance is a per-task folder: a markdown spec (query + hidden requirements checklist, ≥ 12 verifiable items + trajectory requirements) plus synthetic input materials (CSV/documents). No real personal data.
- **Splits.** Load-bearing `train/` (warmup, drives evolution) vs. `test/` (holdout, paired Base-vs-Loaded contrast); requirements overlap $\sim 75\%/25\%$ by design.
- **Collection.** Expert-authored against an authoring contract (`assets/suite_requirement.md`); materials synthesized per task; produced during 2026 framework development. No human subjects.
- **Preprocessing.** Human-readable markdown compiled into machine-readable suite JSON (Suite) by `python -m src.cli.preprocess`; raw source retained as the released artifact.
- **Uses.** Benchmarking agent memory / skill acquisition / workflow distillation and continual-learning diagnostics under the Base-vs-Loaded protocol. *Not* a raw one-shot capability leaderboard; several scenes are locale-specific and should not be over-generalized.
- **Distribution & maintenance.** Public Hugging Face repository (ModelScope mirror) under **CC BY 4.0**; Croissant metadata [Akhtar et al., 2024] is auto-generated (`conformsTo: mlcommons/croissant 1.1`), with the authoritative file structure documented in the dataset card since EALE is file-structured rather than a single table. Versioned updates and corrections are released via the repository git history.

References

- Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, et al. Croissant: A metadata format for ml-ready datasets. In *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning (DEEM)*, 2024. doi: 10.1145/3650203.3663326.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025.
- Hui Chen, Miao Xiong, Yujie Lu, et al. Mlr-bench: Evaluating ai agents on open-ended machine learning research, 2025.

- Jiaju Chen, Yuxuan Lu, Xiaojie Wang, Huimin Zeng, Jing Huang, Jiri Gesi, Ying Xu, Bingsheng Yao, and Dakuo Wang. Multi-agent-as-judge: Aligning llm-agent-based automated evaluation with multi-dimensional human evaluation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 17389–17416, 2026. ACL 2026 Long Papers; also known as MAJ-Eval.
- Comet ML. Opik: Open-source llm evaluation and observability platform. <https://github.com/comet-ml/opik>, 2025. GitHub repository.
- Confident AI. Deepeval: The llm evaluation framework. <https://github.com/confident-ai/deepeval>, 2025. GitHub repository.
- Yuchuan Fu, Xiaohan Yuan, and Dongxia Wang. Ras-eval: A comprehensive benchmark for security evaluation of llm agents in real-world environments, 2025.
- Divyansh Garg, Shaun VanWeelden, Diego Caples, et al. Real: Benchmarking autonomous agents on deterministic simulations of real websites, 2025. NeurIPS 2025 (poster).
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets, 2021. *Communications of the ACM*, 64(12).
- Sihang Jiang, Lipeng Ma, Zhonghua Hong, et al. Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment, 2026.
- Carlos E. Jimenez, John Yang, Alexander Wettig, et al. Swe-bench: Can language models resolve real-world github issues?, 2023. ICLR 2024.
- Alibek T. Kaliyev and Artem Maryanskyy. Beyond task completion: A verification-vs.-conformance gap in tool-evolving agents, 2026. Introduces the EvolveTool-Bench diagnostic benchmark.
- Thomas Kuntz, Agatha Duzan, Hao Zhao, et al. Os-harm: A benchmark for measuring safety of computer use agents, 2025. NeurIPS 2025 Datasets and Benchmarks Track.
- Langfuse. Langfuse: Open-source llm engineering platform. <https://github.com/langfuse/langfuse>, 2025. GitHub repository.
- Xiangyi Li, Wenbo Chen, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026.
- Xiao Liu, Hao Yu, Hanchen Zhang, et al. Agentbench: Evaluating llms as agents, 2023. ICLR 2024.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants, 2023.
- Promptfoo. Promptfoo: Test and evaluate llm outputs. <https://github.com/promptfoo/promptfoo>, 2025. GitHub repository.
- Yunjia Qi, Hao Peng, Xiaozhi Wang, et al. Agentif: Benchmarking instruction following of large language models in agentic scenarios, 2025.
- Jiahao Qiu, Xinzhe Juan, Yimin Wang, et al. Agentdistill: Training-free agent distillation with generalizable mcp boxes, 2025.
- Jialin Song, Xiaodong Liu, Weiwei Yang, et al. Multibreak: A scalable and diverse multi-turn jailbreak benchmark for evaluating llm safety, 2026.
- Yuyao Wang, Zhongjian Zhang, Mo Chi, et al. Evomembench: Benchmarking agent memory from a self-evolving perspective, 2026.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, et al. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory, 2025.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

Junhao Zheng, Xidi Cai, Qiuke Li, et al. Lifelongagentbench: Evaluating llm agents as lifelong learners, 2025.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, et al. Establishing best practices for building rigorous agentic benchmarks, 2025. Introduces the Agentic Benchmark Checklist (ABC).

Mingchen Zhuge, Changsheng Zhao, et al. Agent-as-a-judge: Evaluate agents with agents, 2024.

Table 16: What each evaluated runtime actually does to evolve. “Hook” names the overridden method; “no-op” means the runtime performs no explicit distillation and the delta carries only whatever the agent happened to write to disk during warmup. Mechanism class: **C** = carry-only (no distillation), **D** = per-task distillation, **R** = explicit post-hoc reflection.

Runtime	Class	Hook @ timing → trigger	Delta-committed state
openclaw	C	evolve_after_warmup is a no-op; no explicit evolve at all	/root/.openclaw/workspace/memory, skill-workshop (as written)
openclaw +openspace	C	Same no-op evolve as openclaw; OpenSpace only adds an MCP skill-hub <i>tool surface</i> (retrieval structure), no distillation step	same, plus OpenSpace-local skills
openclaw +agentmemory	C	No-op evolve. The :3111 agentmemory server runs <i>without</i> an LLM provider, so its consolidate/reflect engine is inert — only raw observations are stored	+ /root/.agentmemory (undistilled)
hermes	D	evolve_after_task @ <i>per task</i> : after each warmup task the work runner runs a <i>blocking</i> background review that distills the session into compact skills/memories (end_session() must return clean); evolve_after_warmup is a no-op	/opt/hermes-state/{skills,memories}
hermes +openspace	D	Same per-task review as hermes; OpenSpace registered as an MCP tool	same
hermes +agentmemory	D	Same per-task review; memory routed through the :3111 agentmemory backend	+ /root/.agentmemory
genericagent	C	evolve_after_warmup is a no-op; memory files written to /opt/GenericAgent/memory during warmup are carried implicitly	/opt/GenericAgent/memory
genericagent +active	R	Two extra reflection passes: an <i>isolated</i> GenericAgent chat after each task (evolve_after_task) and one after the whole batch (evolve_after_warmup), each instructed to write layered memory to /opt/GenericAgent/memory	same
evoscientist	C	evolve_after_warmup is a LIFT no-op; but EvoScientist runs an <i>always-on</i> internal memory loop that, during warmup, writes synthesized profiles (SOUL, USER_PROFILE, RESEARCH_TASK, per-project PROJECT_PROFILE) alongside raw observations. No explicit AutoSkills distillation fires (autoskills/proposals stays empty)	/root/.evoscientist/{memories,sessions.db}
evoscientist +active	R	evolve_after_warmup @ <i>post-batch</i> : after all warmup tasks it runs EvoScientist’s AutoSkills maintenance graph in the still-live container (host-injected via docker exec), distilling accumulated observations into explicit skill proposals (autoskills/proposals populated) before the commit; the always-on internal loop still runs underneath	same as evoscientist, plus populated autoskills
prime-agent +active	D	evolve_after_task @ <i>per task</i> : after each warmup task a /refine -global pass distills that trajectory into the global Continual Harness (invoked via -c), carried by docker commit	/root/.prime/agent/ (global harness)